

Section A — CRUD Operations on books Collection

Q1. Insert at least 5 books with fields: title, author, genre, year, copies.

Create the collection and insert multiple book documents using insertMany().

► INPUT

```
use LibraryDB

db.books.insertMany([
  {
    title   : "The Art of Data",
    author  : "Ananya Mehta",
    genre   : "Technology",
    year    : 2019,
    copies  : 6
  },
  {
    title   : "Clean Code Principles",
    author  : "Rohan Desai",
    genre   : "Technology",
    year    : 2021,
    copies  : 4
  },
  {
    title   : "Wings of Fire",
    author  : "A.P.J. Abdul Kalam",
    genre   : "Biography",
    year    : 1999,
    copies  : 10
  },
  {
    title   : "Cosmos",
    author  : "Carl Sagan",
    genre   : "Science",
    year    : 1980,
    copies  : 3
  },
  {
    title   : "Data Structures Simplified",
    author  : "Priya Kulkarni",
    genre   : "Technology",
    year    : 2023,
    copies  : 0
  },
  {
    title   : "The Midnight Library",
    author  : "Matt Haig",
    genre   : "Fiction",
    year    : 2020,
    copies  : 7
  }
])
```

```
}  
  ])  
}
```

◀ OUTPUT

```
{  
  acknowledged : true,  
  insertedIds   : {  
    '0' : ObjectId('64f3a1b2c3d4e5f6a7b8c9d0'),  
    '1' : ObjectId('64f3a1b2c3d4e5f6a7b8c9d1'),  
    '2' : ObjectId('64f3a1b2c3d4e5f6a7b8c9d2'),  
    '3' : ObjectId('64f3a1b2c3d4e5f6a7b8c9d3'),  
    '4' : ObjectId('64f3a1b2c3d4e5f6a7b8c9d4'),  
    '5' : ObjectId('64f3a1b2c3d4e5f6a7b8c9d5')  
  }  
}
```

Q2. Find all books written by a specific author.

Retrieve all documents where author matches "Priya Kulkarni".

▶ INPUT

```
db.books.find(  
  { author: "Priya Kulkarni" }  
)<pre>.pretty()
```

◀ OUTPUT

```
[  
  {  
    _id      : ObjectId('64f3a1b2c3d4e5f6a7b8c9d4'),  
    title    : 'Data Structures Simplified',  
    author   : 'Priya Kulkarni',  
    genre    : 'Technology',  
    year     : 2023,  
    copies   : 0  
  }  
]
```

Q3. Update the number of copies for a given title.

Use `updateOne()` with the `$set` operator to update copies for "The Art of Data".

► INPUT

```
db.books.updateOne(
  { title: "The Art of Data" },
  { $set: { copies: 12 } }
)
```

◄ OUTPUT

```
{
  acknowledged : true,
  matchedCount  : 1,
  modifiedCount : 1,
  upsertedId    : null
}
```

Q4. Delete a book that has zero copies.

Use `deleteOne()` to remove the first document where copies equals 0.

► INPUT

```
db.books.deleteOne(
  { copies: 0 }
)
```

◄ OUTPUT

```
{
  acknowledged : true,
  deletedCount  : 1
}
```

Q5. Find all books published after 2015.

Use the comparison operator *\$gt* on the *year* field.

► INPUT

```
db.books.find(  
  { year: { $gt: 2015 } }  
)
```

◀ OUTPUT

```
[  
  {  
    _id    : ObjectId('64f3a1b2c3d4e5f6a7b8c9d0'),  
    title  : 'The Art of Data',  
    author : 'Ananya Mehta',  
    genre  : 'Technology',  
    year   : 2019,  
    copies : 12  
  },  
  {  
    _id    : ObjectId('64f3a1b2c3d4e5f6a7b8c9d1'),  
    title  : 'Clean Code Principles',  
    author : 'Rohan Desai',  
    genre  : 'Technology',  
    year   : 2021,  
    copies : 4  
  },  
  {  
    _id    : ObjectId('64f3a1b2c3d4e5f6a7b8c9d5'),  
    title  : 'The Midnight Library',  
    author : 'Matt Haig',  
    genre  : 'Fiction',  
    year   : 2020,  
    copies : 7  
  }  
]
```

Q6. Group books by genre and count total books in each genre.

Use the aggregate() pipeline with \$group and \$sum.

► INPUT

```
db.books.aggregate([
  {
    $group: {
      _id      : "$genre",
      totalBooks : { $sum: 1 }
    }
  }
])
```

◀ OUTPUT

```
[
  { _id: 'Technology', totalBooks: 2 },
  { _id: 'Biography',  totalBooks: 1 },
  { _id: 'Science',    totalBooks: 1 },
  { _id: 'Fiction',    totalBooks: 1 }
]
```

Q7. Find top 3 most recent books.

Sort by year descending and limit the result to 3 documents.

► INPUT

```
db.books.find().sort({ year: -1 }).limit(3).pretty()
```

◀ OUTPUT

```
[
  {
    _id    : ObjectId('64f3a1b2c3d4e5f6a7b8c9d1'),
    title  : 'Clean Code Principles',
    author : 'Rohan Desai',
    genre  : 'Technology',
    year   : 2021,
    copies : 4
  },
  {
    _id    : ObjectId('64f3a1b2c3d4e5f6a7b8c9d5'),
    title  : 'The Midnight Library',
    author : 'Matt Haig',
    genre  : 'Fiction',
    year   : 2020,
    copies : 7
  },
  {
    _id    : ObjectId('64f3a1b2c3d4e5f6a7b8c9d0'),
    title  : 'The Art of Data',
    author : 'Ananya Mehta',
    genre  : 'Technology',
    year   : 2019,
    copies : 12
  }
]
```

Q8. Count how many books each author has published.

Group by author and count, then sort descending.

► INPUT

```
db.books.aggregate([
  {
    $group: {
      _id      : "$author",
      bookCount : { $sum: 1 }
    }
  },
  { $sort: { bookCount: -1 } }
])
```

◄ OUTPUT

```
[
  { _id: 'Ananya Mehta',      bookCount: 1 },
  { _id: 'Rohan Desai',      bookCount: 1 },
  { _id: 'A.P.J. Abdul Kalam', bookCount: 1 },
  { _id: 'Carl Sagan',       bookCount: 1 },
  { _id: 'Matt Haig',        bookCount: 1 }
]
```

Q9. Search books whose title contains the word "data".

Use a regular expression with the \$regex operator (case-insensitive).

► INPUT

```
db.books.find(
  { title: { $regex: "data", $options: "i" } }
).pretty()
```

◄ OUTPUT

```
[
  {
    _id      : ObjectId('64f3a1b2c3d4e5f6a7b8c9d0'),
    title    : 'The Art of Data',
    author   : 'Ananya Mehta',
    genre    : 'Technology',
    year     : 2019,
    copies   : 12
  }
]
```